
机器学习编程实验内容

一、实验目的：

1. 熟悉MATLAB, PYTHON等机器学习编程开发环境
2. 掌握基本的机器学习运算编程方法
3. 能使用真实数据集建立机器学习算法模型

二、实验内容：

- 1 实验一 机器学习开发编程基本运算熟悉(MATLAB为例)，使用PYTHON可参考，实验课-anaconda下载安装及python spyder使用文件
- 2 实验二 线性模型和最小二乘法（编程语言不限）
- 3 实验三 梯度下降法（编程语言不限）
- 4 实验四 BP神经网络实验（编程语言不限）

三、实验考核要求：

完成2次实验报告，从实验二三四中任选2次（参考实验报告模板）

实验一 机器学习开发编程基本运算熟悉(MATLAB为例)

一、实验目的：

1. 熟悉MATLAB开发环境
2. 掌握矩阵、变量、表达式的各种基本运算

内容一：实验基本知识

内容二：MATLAB数值及符号运算

内容三：MATLAB语言的程序设计

内容四：MATLAB的图形绘制

二、实验基本知识：

1.熟悉MATLAB环境:

MATLAB桌面和命令窗口、命令历史窗口、帮助信息浏览器、工作空间浏览器

文件和搜索路径浏览器。

2.掌握 MATLAB 常用命令

clc	清除命令窗口中内容
clear	清除工作空间中变量
help	对所选函数的功能、调用格式及相关函数给出说明
lookfor	查找具有某种功能的函数但却不知道该函数的准确名称
who	查询工作空间中的变量信息

3.MATLAB变量与运算符

变量命名规则如下：

- (1) 变量名可以由英语字母、数字和下划线组成
- (2) 变量名应以英文字母开头
- (3) 长度不大于31个
- (4) 区分大小写

MATLAB中设置了一些特殊的变量与常量，列于下表。

表1 MATLAB的特殊变量与常量

变量名	功能说明	变量名	功能说明
ANS	默认变量名，以应答最近一次操作运算结果	realmin	最小的正实数
i或j	虚数单位	INF(inf)	无穷大
pi	圆周率	NAN(nan)	不定值 (0/0)
eps	浮点数的相对误差	nargin	函数实际输入参数个数
realmax	最大的正实数	nargout	函数实际输出参数个数

MATLAB运算符，通过下面几个表来说明MATLAB的各种常用运算符

表 2 MATLAB 算术运算符

操作符	功能说明	操作符	功能说明
+	加	\	矩阵左除
-	减	.\	数组左除

*	矩阵乘	/	矩阵右除
.*	数组乘	./	数组右除
^	矩阵乘方	'	矩阵转置
.^	数组乘方	.'	数组转置

表 3 MATLAB 关系运算符

操作符	功能说明
==	等于
~=	不等于
>	大于
<	小于
>=	大于等于
<=	小于等于

表 4 MATLAB 逻辑运算符

逻辑运算符	逻辑运算	说明
&	And	逻辑与
	Or	逻辑或
~	Not	逻辑非
Xor		逻辑异或

表 5 MATLAB 特殊运算

符号	功能说明示例	符号	功能说明
----	--------	----	------

			示例
:	1:1:4;1:2:11	.	
;	分隔行	..	
,	分隔列	...	
()		%	注释
[]	构成向量、矩阵	!	调用操作 系统命令
{}	构成单元数组	=	用于赋值

4.MATLAB的一维、二维数组的寻访

表6 子数组访问与赋值常用的相关指令格式

指令格式	指令功能
$A(r,c)$	数组 A 中 r 指定行、 c 指定列之元素组成的子数组
$A(r,:)$	数组 A 中 r 指定行对应的所有列之元素组成的子数组
$A(:,c)$	数组 A 中 c 指定列对应的所有行之元素组成的子数组
$A(:)$	数组 A 中各列元素首尾相连组成的“一维长列”子数组
$A(i)$	"一维长列"子数组中的第 i 个元素
$A(r,c)=Sa$	数组 A 中 r 指定行、 c 指定列之元素组成的子数组的赋值
$A(:)=D(:)$	数组全元素赋值，保持 A 的行宽、列长不变， A 、 D 两组元素 总和应相同

5.MATLAB的基本运算

表 7 两种运算指令形式和实质内涵的异同表

数组运算		矩阵运算	
指令	含义	指令	含义
$A.'$	非共轭转置	A'	共轭转置
$A=s$	把标量s赋给A的每个元素		
$s+B$	标量s分别与B元素之和		
$s-B, B-s$	标量s分别与B元素之差		
$s.*A$	标量s分别与A元素之积	$s*A$	标量s分别与A每个元素之积
$s./B, B.\backslash s$	标量s分别被B的元素除	$S*inv(B)$	B阵的逆乘s
$A.^n$	A的每个元素自乘n次	A^n	A阵为方阵，自乘n次
$A.^p$	对A各元素分别求非整数幂	A^p	方阵A的非整数乘方
$A+B$	对应元素相加	$A+B$	矩阵相加
$A-B$	对应元素相减	$A-B$	矩阵相减
$A.*B$	对应元素相乘	$A*B$	内维相同矩阵相乘
$A./B$	A的元素别B的对应元素除	A/B	A右除B
$B.\backslash A$	与上相同	$B\backslash A$	A左除B
$exp(A)$	以自然数e为底，分别以A的元素为指数，求幂	$expm(A)$	A的矩阵指数函数
$log(A)$	对A的各元素求对数	$logm(A)$	A的矩阵对数函数
$sqrt(A)$	对A的各元素求平方根	$sqrtm(A)$	A的矩阵平方根函数

6.MATLAB的常用函数

表8 标准数组生成函数

指令	含义	指令	含义
diag	产生对角形数组 (对高维不适用)	rand	产生均匀分布随机数组
eye	产生单位数组 (对高维不适用)	randn	产生正态分布随机数组
magi c	产生魔方数组 (对高维不适用)	zeros	产生全0数组
ones	产生全1数组	size()	返回指定矩阵的行数和 列数

表9 数组操作函数

指令	含义
diag	提取对角线元素，或生成对角阵
flipud	以数组“水平中线”为对称轴，交换上下对称位置 上的数组元素
fliplr	以数组“垂直中线”为对称轴，交换左右对称位置 上的数组元素
reshape	在总元素数不变的前提下，改变数组的“行数、 列数”
rot90	矩阵逆时针旋转90度
det	方阵的行列式值

rank	矩阵的秩
------	------

三、实验内容

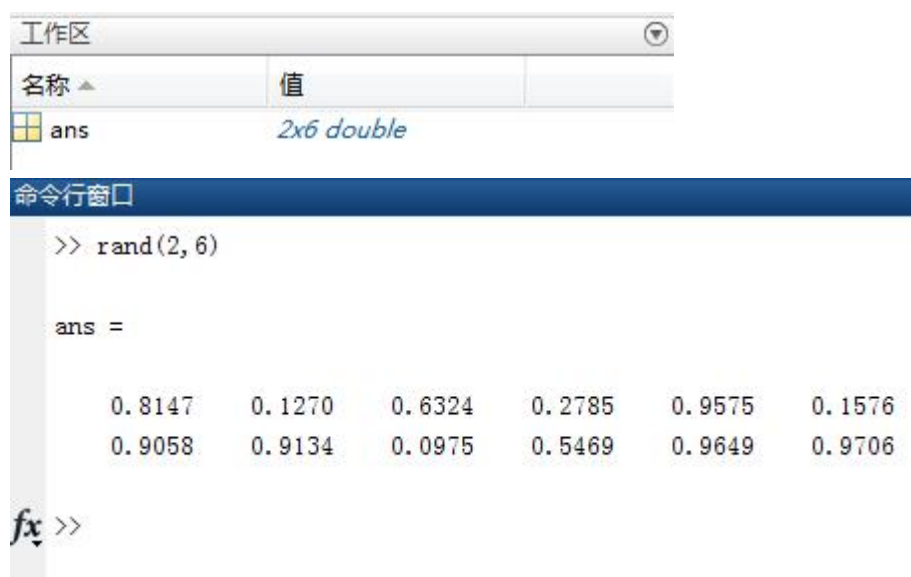
- 1、学习安装MATLAB软件。
- 2、学习使用help命令，例如在命令窗口输入help eye，然后根据帮助说明，学习使用指令eye（其它不会用的指令，依照此方法类推）
- 3、学习使用clc、clear，观察command window、command history和workspace等窗口的变化结果。
- 4、初步程序的编写练习，新建M-file，保存（自己设定文件名，例如exerc1、exerc2、exerc3……），学习使用MATLAB的基本运算符、数组寻访指令、标准数组生成函数和数组操作函数。

注意：每一次**M-file**的修改后，都要存盘。

四、实验结果

练习A：

- (1) help rand，然后随机生成一个 2×6 的数组，观察command window、command history和workspace等窗口的变化结果。



(2) 学习使用clc、clear，了解其功能和作用。

(3) 输入C=1:2:20，则C (i) 表示什么？其中i=1,2,3,...,10。

```
命令行窗口
>> C=1:2:20

C =

     1     3     5     7     9    11    13    15    17    19

>> C(4)

ans =

     7

>> C(2)

ans =

     3

>> C(6)

ans =

    11

fx >> |
```

(4) 输入A=[7 1 5; 2 5 6; 3 1 5]，B=[1 1 1; 2 2 2; 3 3 3]，在命令窗口

中执行下列表达式，掌握其含义：

A(2, 3) A(:,2) A(3,:) A(:,1:2:3) A(:,3).*B(:,2) A(:,3)*B(2,:)

A*B A.*B A^2 A.^2 B/A B./A

```
>> A=[7 1 5;2 5 6;3 1 5]
```

```
A =
```

```
7     1     5
2     5     6
3     1     5
```

```
>> B=[1 1 1; 2 2 2; 3 3 3]
```

```
B =
```

```
1     1     1
2     2     2
3     3     3
```

```
>> |
```

```
>> A(:,1:2:3)
```

```
ans =
```

```
7     5
2     6
3     5
```

```
>> A(:,3).*B(:,2)
```

```
ans =
```

```
5
12
15
```

```
>> A(:,3)*B(2,:)
```

```
ans =
```

```
10    10    10
12    12    12
10    10    10
```

```
>>
```

```
>> A(2,3)
```

```
ans =
```

```
6
```

```
>> A(:,2)
```

```
ans =
```

```
1
5
1
```

```
>> A(3,:) 
```

```
ans =
```

```
3     1     5
```

```
>> |
```

```
>> A*B
```

```
ans =
```

```
24    24    24
30    30    30
20    20    20
```

```
>> A.*B
```

```
ans =
```

```
7     1     5
4    10    12
9     3    15
```

```
>> A^2
```

```
ans =
```

```
66    17    66
42    33    70
38    13    46
```

```
>>
```

```
>> A.^2
```

```
ans =
```

```
    49     1    25
     4    25    36
     9     1    25
```

```
>> B/A
```

```
ans =
```

```
    0.1842    0.2105   -0.2368
    0.3684    0.4211   -0.4737
    0.5526    0.6316   -0.7105
```

```
>> B./A
```

```
ans =
```

```
    0.1429    1.0000    0.2000
    1.0000    0.4000    0.3333
    1.0000    3.0000    0.6000
```

```
>> |
```

(5) 二维数组的创建和寻访，创建一个二维数组（4×8）A，查询数组A

第2行、第3列的元素，查询数组A第2行的所有元素，查询数组A

第 6 列 的 所 有 元 素 。

```
>> A=[1 2 3 4 5 6 7 8;8 7 6 5 4 3 2 1;11 22 33 44 55 66 77 88;88 77 66 55 44 33 22 11]
```

```
A =
```

```
     1     2     3     4     5     6     7     8
     8     7     6     5     4     3     2     1
    11    22    33    44    55    66    77    88
    88    77    66    55    44    33    22    11
```

```
>> A(2,3)
```

```
ans =
```

```
     6
```

```
>>
```

```
>> A(2,:)
```

```
ans =
```

```
8    7    6    5    4    3    2    1
```

```
>> A(:,6)
```

```
ans =
```

```
6
3
66
33
```

```
>> |
```

(6) 两种运算指令形式和实质内涵的比较。设有3个二维数组 $A_{2 \times 4}$, $B_{2 \times 4}$,

$C_{2 \times 2}$, 写出所有由2个数组参与的合法的数组运算和矩阵指令。

```
>> A=randn(2,4)
```

```
A =
```

```
0.6007   -1.1135    1.5326    0.3714
-1.2141   -0.0068   -0.7697   -0.2256
```

```
>> B=randn(2,4)
```

```
B =
```

```
1.1174    0.0326    1.1006    0.0859
-1.0891    0.5525    1.5442   -1.4916
```

```
>> C=randn(2,2)
```

```
C =
```

```
-0.7423    2.3505
-1.0616   -0.6156
```

```
>> A+B
```

```
ans =
```

```
1.7181   -1.0809    2.6332    0.4573
-2.3032    0.5457    0.7745   -1.7172
```

```
>> A\C
```

```
ans =
```

```
1.5720   -0.6190
0         0
-1.1005    1.7762
0         0
```

```
>> |
```

(7) 学习使用表8列的常用函数（通过help方法）

```
>> magic(6)

ans =

    35     1     6    26    19    24
     3    32     7    21    23    25
    31     9     2    22    27    20
     8    28    33    17    10    15
    30     5    34    12    14    16
     4    36    29    13    18    11
```

```
>> zeros(5)

ans =

     0     0     0     0     0
     0     0     0     0     0
     0     0     0     0     0
     0     0     0     0     0
     0     0     0     0     0
```

```
>> |
```

(8) 学习使用表9数组操作函数。

```
>> A=magic(5)
```

```
A =

    17    24     1     8    15
    23     5     7    14    16
     4     6    13    20    22
    10    12    19    21     3
    11    18    25     2     9

>> flipud(A)

ans =

    11    18    25     2     9
    10    12    19    21     3
     4     6    13    20    22
    23     5     7    14    16
    17    24     1     8    15
```

```
>> C=diag(A)
```

```
C =

    17
     5
    13
    21
     9

>> rot90(A)

ans =

    15    16    22     3     9
     8    14    20    21     2
     1     7    13    19    25
    24     5     6    12    18
    17    23     4    10    11

>>
```

练习B

(9) 用reshape指令生成下列矩阵（不超过3次操作），并取出方框内的数组元素。

```

1  2  3  4  5
10 9  8  7  6
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25

>> A=1:25;
>> C=reshape(A, [5, 5])

C =

     1     6    11    16    21
     2     7    12    17    22
     3     8    13    18    23
     4     9    14    19    24
     5    10    15    20    25

>> I=C'

I =

     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
    16    17    18    19    20
    21    22    23    24    25

>> Z=I(2, [3, 4])

Z =

     8     9

>> N=I([2, 3, 4], 4)

N =

     9
    14
    19

>> R=I([4, 5], [1, 2, 3])

R =

    16    17    18
    21    22    23

>>

```

二 MATLAB数值及符号运算

一、 实验目的：

- 1、掌握矩阵的基本运算
- 2、掌握矩阵的数组运算
- 3、掌握多项式的基本运算
- 4、会求解代数方程
- 5、掌握创建符号表达式和矩阵的方法

6、掌握符号表达式的微分和积分运算

二、 实验基本知识:

1、多项式运算

poly —— 产生特征多项式系数向量

roots —— 求多项式的根

p=poly2str(c,'x') (以习惯方式显示多项式)

conv, convs 多项式乘运算

deconv 多项式除运算

多项式微分

polyder(p): 求p的微分

polyder(a,b): 求多项式a,b乘积的微分

2、代数方程组求解

1) 恰定方程组的解

方程 $ax=b$ (a为非奇异)两种求解方法: $x=\text{inv}(a)*b$ — 采用求逆运算解方程

$x=a\backslash b$ — 采用左除运算解方程

2) 超定方程组的解

方程 $ax=b$, $m < n$ 时此时不存在唯一解。 $x=(a' a)^{-1} a' b$ —— 求逆法

3) 欠定方程组的解

matlab可求出两个解: 用除法求的解x是具有最多零元素的解

是具有最小长度或范数的解, 这个解是基于伪逆pinv求得的。

3、符号矩阵的创建

1) 用matlab函数sym创建矩阵 (symbolic 的缩写)

命令格式: `A=sym('[                      ]')`

2) 用字符串直接创建矩阵

4、符号矩阵与数值矩阵的转换

将数值矩阵转化为符号矩阵, 函数调用格式: `double(A)`

将符号矩阵转化为数值矩阵, 函数调用格式: `numeric(A)`

5、符号微积分与积分变换

`diff(f)` — 对缺省变量求微分

`diff(f,v)` — 对指定变量v求微分

`diff(f,v,n)` — 对指定变量v求n阶微分

`int(f)` — 对f表达式的缺省变量求积分

`int(f,v)` — 对f表达式的v变量求积分

`int(f,v,a,b)` — 对f表达式的v变量在 (a,b)区间求定积分

6、符号代数方程求解

`solve(f)` —— 求一个方程的解

`solve(f1,f2, ...fn)` —— 求n个方程的解

7、符号微分方程求解指令: `dsolve`

命令格式: `dsolve(f,g):` `f` —— 微分方程, 可多至12个微分方程的求解; `g`为初始条件, 默认自变量为 '`x`',可任意指定自变量'`t`', '`u`'等, 微分方程的各阶导数项以大写字母D表示

三、实验内容及结果:

1、已知 $a=[1\ 2\ 3;4\ 5\ 6;7\ 8\ 0]$ ，求其特征多项式并求其根。

```
>> a=[1 2 3;4 5 6;7 8 0]
```

a =

1	2	3
4	5	6
7	8	0

```
>> p=poly2str(a,'x')
```

p =

$x^2 + 4x + 7$

```
>> a=[1 2 3 4 5 6 7 8 0];
```

```
>> b=roots(a)
```

b =

0.0000 + 0.0000i
-1.3995 + 0.0000i
-1.0164 + 0.9455i
-1.0164 - 0.9455i
-0.0946 + 1.3479i
-0.0946 - 1.3479i
0.8107 + 0.9836i
0.8107 - 0.9836i

2、已知多项式 $a(x)=x^2+2x+3$ ， $b(x)=4x^2+5x+6$ ，求a，b的积并微分。

```
>> a=[1 2 3];
```

```
>> b=[4 5 6];
```

```
>> c=polyder(a,b)
```

c =

16	39	56	27
----	----	----	----

3、求解方程

1) $\begin{cases} x_1 + 2x_2 = 8 \\ 2x_1 + 3x_2 = 13 \end{cases}$

2) $\begin{cases} x_1 + 2x_2 = 1 \\ 2x_1 + 3x_2 = 2 \\ 3x_1 + 4x_2 = 3 \end{cases}$

3) $\begin{cases} x_1 + 2x_2 + 3x_3 = 1 \\ 2x_1 + 3x_2 + 4x_3 = 2 \end{cases}$

```

>> a=[1 2;2 3]; >> a=[1 2;2 3;3 4]; >> a=[1 2 3;2 3 4];
>> B=[8 13]'; >> b=[1 2 3]'; >> b=[1 2]';
>> a\B >> a\b >> a\b

ans =          ans =          ans =
      2.0000      1.0000          1
      3.0000      0.0000          0
              0.0000          0

>> >> | >> |

```

4、用两种方法创建符号矩阵， $A = \begin{bmatrix} a & 2*b \\ 3*a & 0 \end{bmatrix}$

$\begin{bmatrix} a & 2*b \\ 3*a & 0 \end{bmatrix}$

```
>> r=sym(' [a,2*b;3*b,0]')
```

```
r =
```

```
[ a, 2*b]
[ 3*b, 0]
```

```
>> syms a b;
```

```
>> t=[a,2*b;3*b,0]
```

```
t =
```

```
[ a, 2*b]
[ 3*b, 0]
```

```
>>
```

5、计算二重不定积分 $\iint x e^{-xy} dx dy$

```
>> syms x y;
```

```
>> z=int(int(x*exp(-1*x*y),x),y)
```

```
z =
```

```
exp(-x*y)/y
```

6、对符号方程 $f = ax^2+bx+c$ 求解。1) 对x求解，2) 对a求解。

```
>> syms a b c x;
[x]=solve('a*x^2+b*x+c=0')

x =

-(c + a*x^2)/b

>> [a]=solve('a*x^2+b*x+c=0')

a =

-(c + a*x^2)/b

>>
```

7、求解微分方程 $\frac{d^2y}{dx^2} + 2\frac{dy}{dx} + 2y = 0, y(0) = 1, \frac{dy}{dx}(0) = 0$ 。

```
命令窗口
>> y=dsolve('D2y+2*Dy+2*y=0','y(0)=1,Dy(0)=0','x')

y =

exp(-x)*cos(x) + exp(-x)*sin(x)

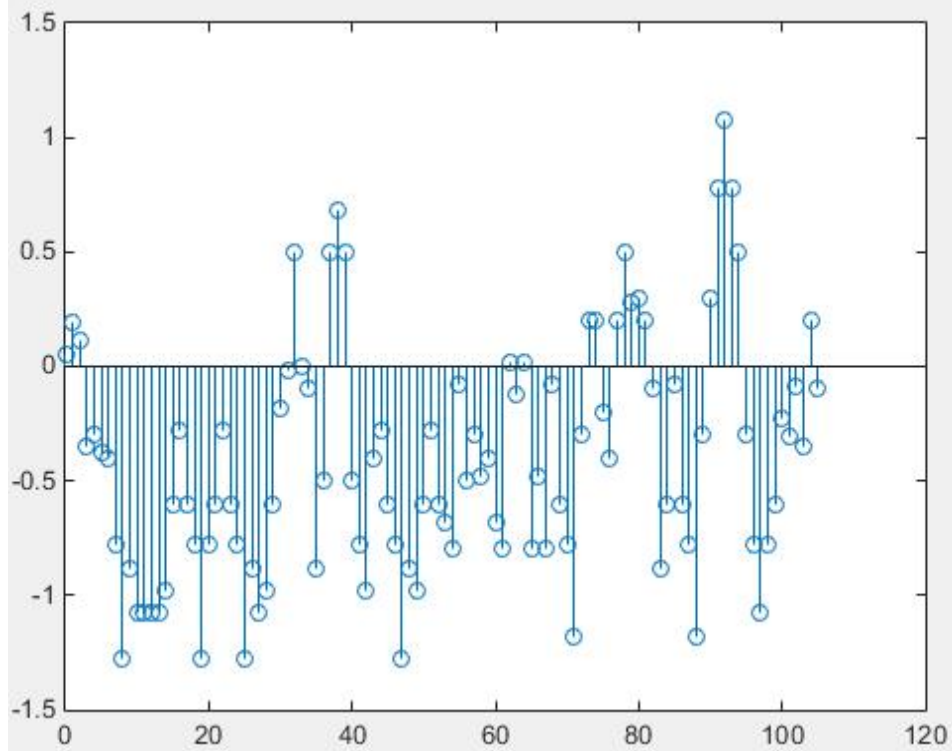
fx >>
```

8、已知系统冲激响应为 $h(n)=[0.05,0.24,0.40,0.24,0.15,-0.1,0.1]$ ，系统输入 $u(n)$ 由指令 `randn('state',1);u=2*(randn(1,100)>0.5)-1` 产生，该输入信号的起始作用时刻为0。试用直杆图（提示：用stem指令）画出分别显示该系统输入、输出信号的两张子图。

命令行窗口

```
>> h=[0.05,0.24,0.40,0.24,0.15,-0.1,0.1];  
>> randn('state',1);  
>> u=2*(randn(1,100)>0.5)-1;  
>> c=conv(h,u);  
>> k=[0:105];  
>> stem(k,c)
```

fx >> |



实验三 MATLAB语言的程序设计

一、实验目的：

- 1、熟悉MATLAB程序编辑与设计环境
- 2、掌握各种编程语句语法规则及程序设计方法
- 3、函数文件的编写和设计
- 4、了解和熟悉跨空间变量传递和赋值

二、实验基本知识：

1、程序流程控制语句

for循环结构

语法：for i = 初值：增量：终值

语句1

.....

语句n

end

说明：1 . i = 初值：终值，则增量为1。

2 . 初值、增量、终值可正可负，可以是整数，也可以是小数，只须符合数学逻辑。

while 循环结构

语法：while 逻辑表达式

循环体语句

end

说明：1、while结构依据逻辑表达式的值判断是否执行循环体语句。若表达式的值

为真，执行循环体语句一次、在反复执行时，每次都要进行判断。若表达式的值为假，则程序执行end之后的语句。

2、为了避免因逻辑上的失误，而陷入死循环，建议在循环体语句的适当位置加break语句、以便程序能正常执行。（执行循环体的次数不确定；每一次执行循环体后，一定会改变while后面所跟关系式的值。）

3、while循环也可以嵌套、其结构如下：

```
while逻辑表达式1
    循环体语句1
while逻辑表达式2
    循环体语句2
end
    循环体语句3
end
```

if-else-end分支结构

```
if 表达式 1
    语句1
else if 表达式 2(可选)
    语句2
else(可选)
    语句3
end
```

end

说明：1.if结构是一个条件分支语句，若满足表达式的条件，则往下执行；若不满足，则跳出if结构。

2. else if表达式2与else为可选项，这两条语句可依据具体情况取舍。

3. 注意：每一个if都对应一个end，即有几个if，记就应有几个end。

switch-case结构

语法：switch表达式

case常量表达式1

语句组1

case常量表达式2

语句组2

.....

otherwise

语句组n

end

说明：1. switch后面的表达式可以是任何类型，如数字、字符串等。

2. 当表达式的值与case后面常量表达式的值相等时，就执行这个case后面的语句组如果所有的常量表达式的值都与这个表达式的值不相等时，则执行otherwise后的执行语句。

程序流程控制指令

break、return、pause

2、函数文件的结构和编写方法

函数只能在函数体内对变量进行操作,也就是只能访问函数本身工作空间中的变量。

M函数文件的结构

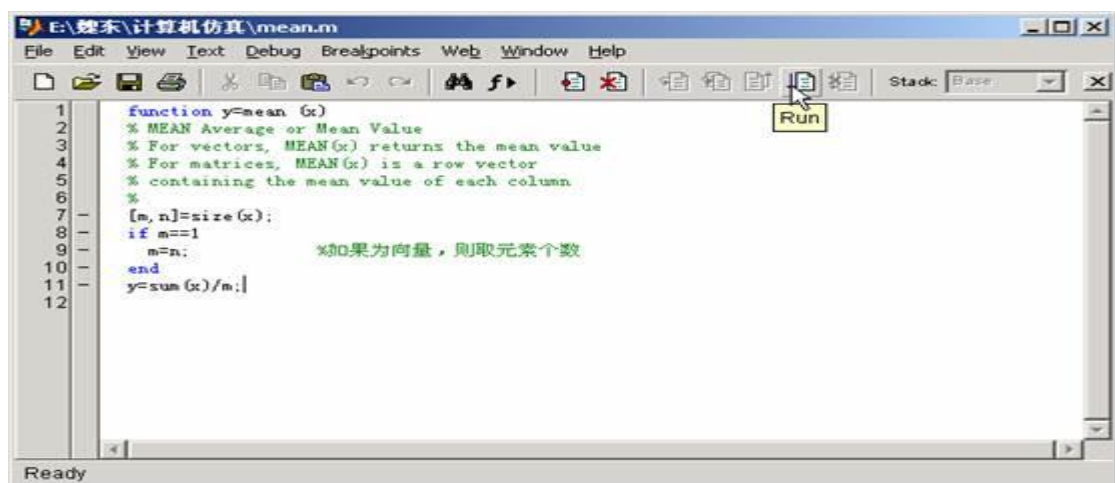
- (1) 函数定义行 (function)
- (2) H1行 (函数帮助文本的第一行)
- (3) 函数帮助文本
- (4) 函数体
- (5) 注释

函数文件编写后,保存时,其文件名必须与函数名相同。注意:函数名不要与MATLAB自身的函数命令相同。

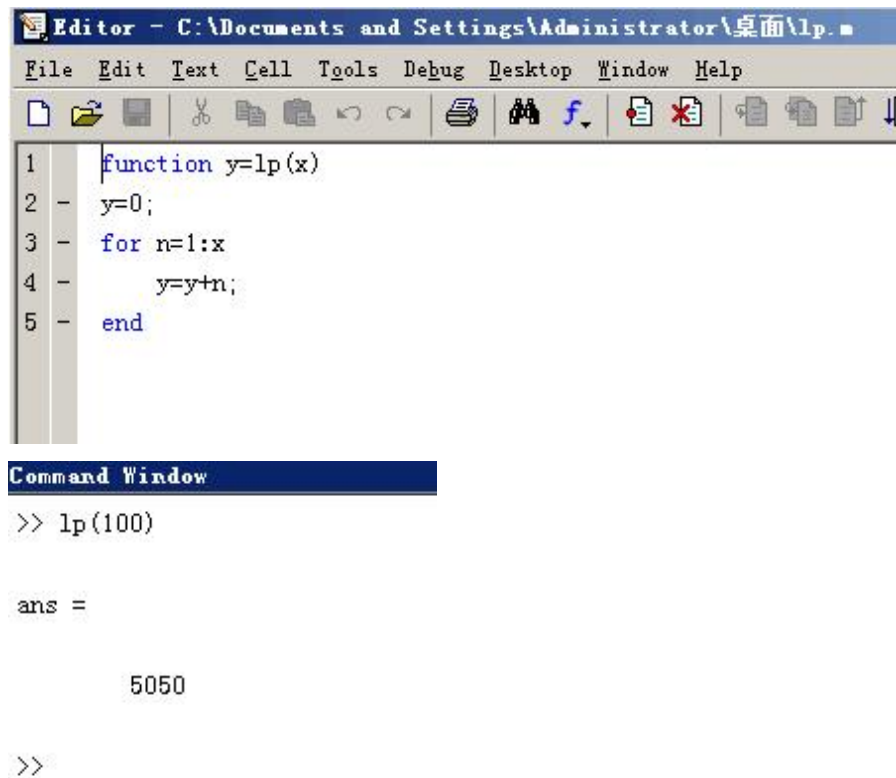
三、实验内容及结果:

练习A

1、熟悉MATLAB程序编辑与设计环境



2、用 for 循环语句实现求 1~100 的和。



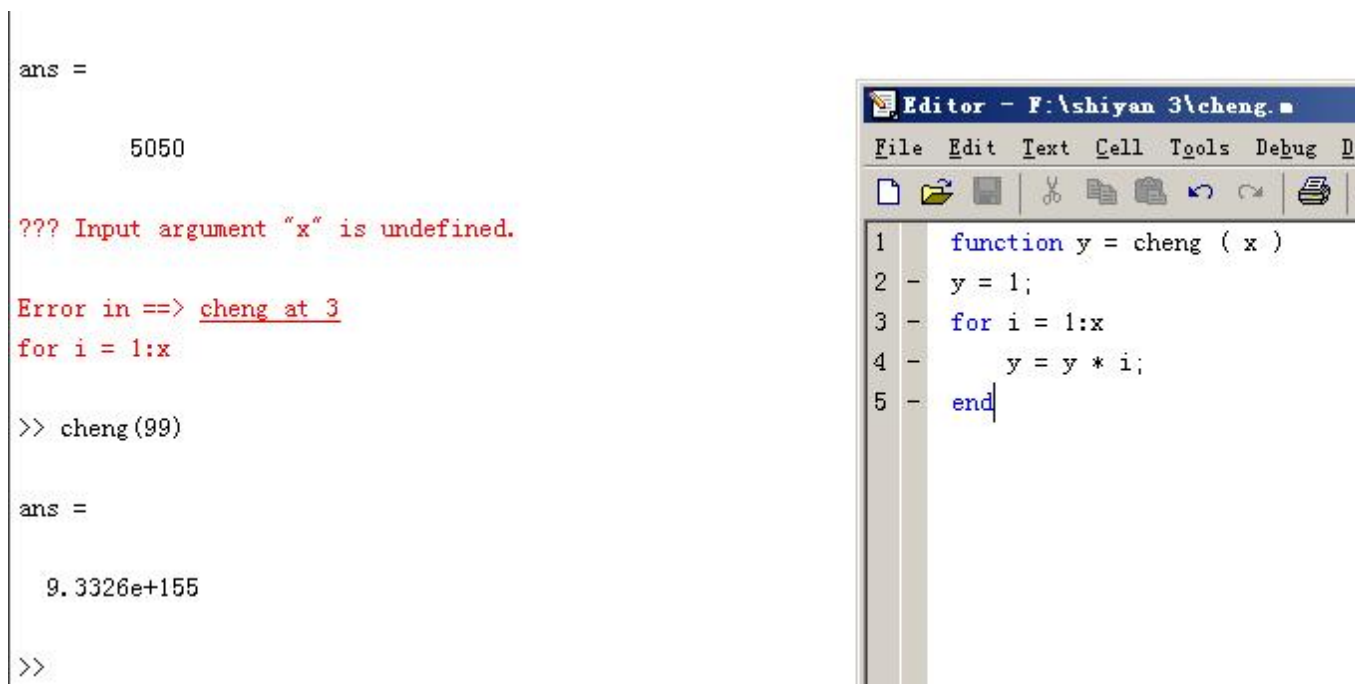
The image shows a MATLAB Editor window titled 'Editor - C:\Documents and Settings\Administrator\桌面\lp.m'. The code in the editor is:

```
1 function y=lp(x)
2 - y=0;
3 - for n=1:x
4 -     y=y+n;
5 - end
```

Below the editor is the Command Window. The command `>> lp(100)` has been entered, and the output is:

```
ans =
      5050
>>
```

3、用 for 循环语句实现编写一个求 n 阶乘的函数文件。



The image shows a MATLAB Command Window on the left and a MATLAB Editor window on the right. The Command Window shows the execution of a function `cheng` with the input 99, resulting in an error and a large number.

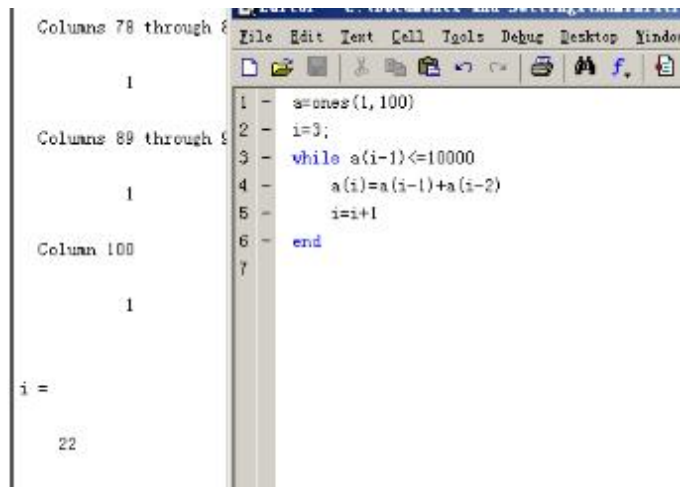
```
ans =
      5050
??? Input argument "x" is undefined.
Error in ==> cheng at 3
for i = 1:x
>> cheng(99)
ans =
  9.3326e+155
>>
```

The Editor window is titled 'Editor - F:\shiyang 3\cheng.m'. The code in the editor is:

```
1 function y = cheng ( x )
2 - y = 1;
3 - for i = 1:x
4 -     y = y * i;
5 - end
```

4、一行 100 列的 Fibonacci 数组 a，元素满足 Fibonacci 规则：a(1)=a(2)=1，

$a(i)=a(i-1)+a(i-2)(i=3,4,\cdots)$ ，求该数组中第一个大于 10000 的元素。



The image shows a MATLAB script and its execution results. The script defines a vector `a` of size 100, initializes `a(1)=1` and `a(2)=1`, and then uses a `while` loop to calculate subsequent Fibonacci numbers until the first value greater than 10000 is found. The command window shows the value of `i` as 22.

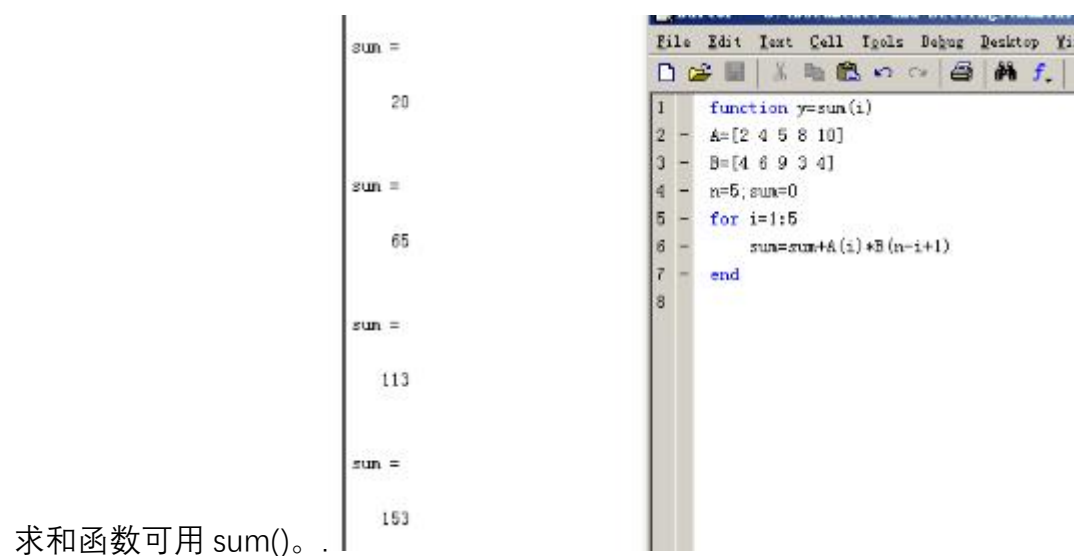
```

1 - a=ones(1,100)
2 - i=3;
3 - while a(i-1)<=10000
4 -     a(i)=a(i-1)+a(i-2)
5 -     i=i+1
6 - end
7
i =
    22

```

练习 B 思考题

4、已知一维数组 $A=[2,4,5,8,10], B=[4,6,9,3,4]$ ，用 for 循环语句实现 $\sum_{i=1}^n A_i B_{n-i+1}$ 。



The image shows a MATLAB script and its execution results. The script defines two vectors `A` and `B`, initializes `n=5` and `sum=0`, and then uses a `for` loop to calculate the dot product of `A` and `B`. The command window shows the final value of `sum` as 153.

```

1 - function y=sum(i)
2 - A=[2 4 5 8 10]
3 - B=[4 6 9 3 4]
4 - n=5; sum=0
5 - for i=1:5
6 -     sum=sum+A(i)*B(n-i+1)
7 - end
8
sum =
    153

```

求和函数可用 `sum()`。

5、利用 for 循环求 $1! + 2! + 3! + \dots + 5!$ 的

```
1 - sum = 0;y=1;
2 - for i= 1:5
3 -     y=y*i;
4 -     sum=sum+y;
5 - end
6 - sum
7
```

值。

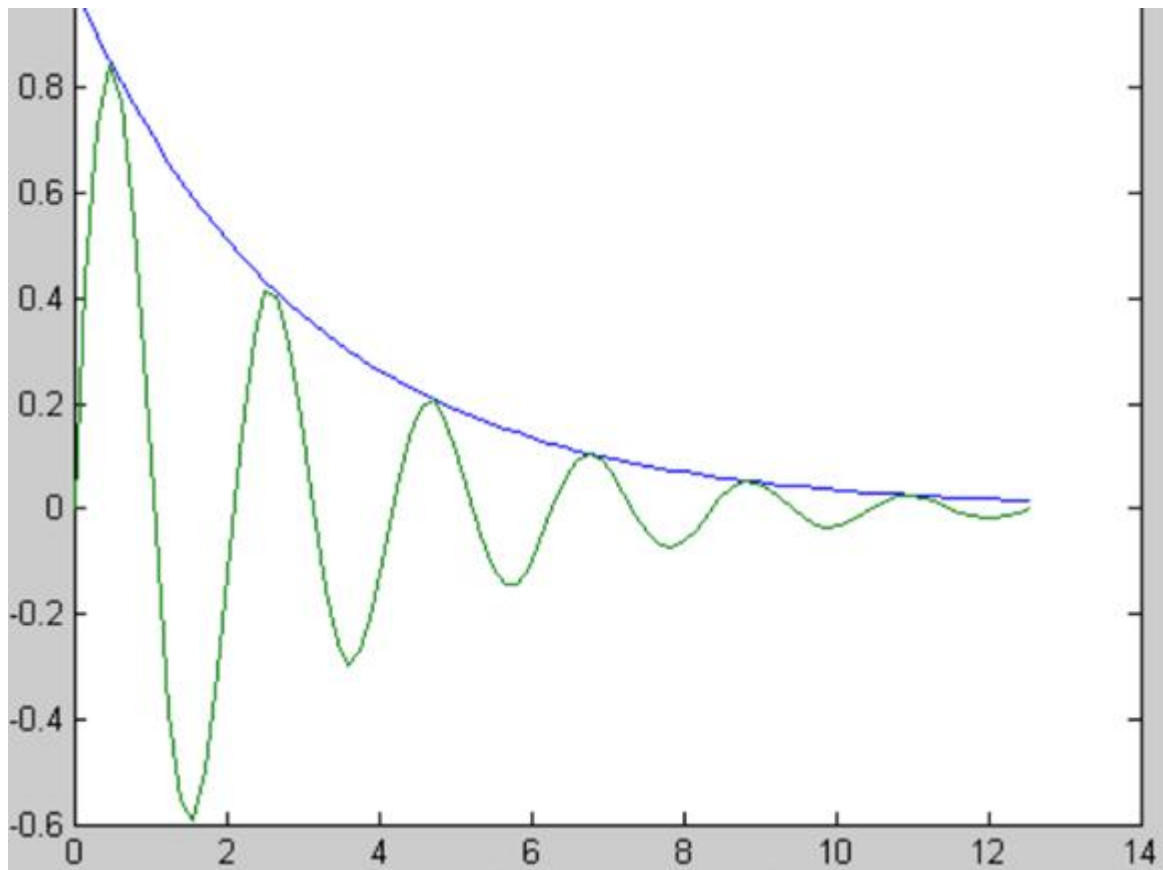
```
sum =
    153
```

6、编写 M 函数表示曲线 $y_2=e^{-t/3}\sin(3t)$ 以及它的包络线 $y_1=e^{-t/3}$ ，并从命令窗口输入命令语句绘制曲线， t 的取值范围是 $[0, 4\pi]$ 。

```
1 function y=fn(t)
2 - y1=exp(-t/3);
3 - y2=exp(-t/3).*sin(3*t);
4 - y=[y1;y2];
```

```
12.5664

>> t=0:pi/20:4*pi;
>> plot(t,fn(t))
```



7、设 $f(x) = \frac{1}{(x-2)^2 + 0.1} + \frac{1}{(x-3)^4 + 0.01}$ ，编写一个 M 函数文件，使得调用 $f(x)$ 时， x 可用矩阵代入，得出的 $f(x)$ 为同阶矩阵。

```
1 function y=f(x)
2 - y=1./((x-2).^2+0.1)+1./((x-3).^4+0.01)
```

```
>> x=rand(2,2);
>> y

y =

    0.7605    0.8145
    0.8622    0.7680
    0.4766    0.7838
    0.8382    0.5323
```

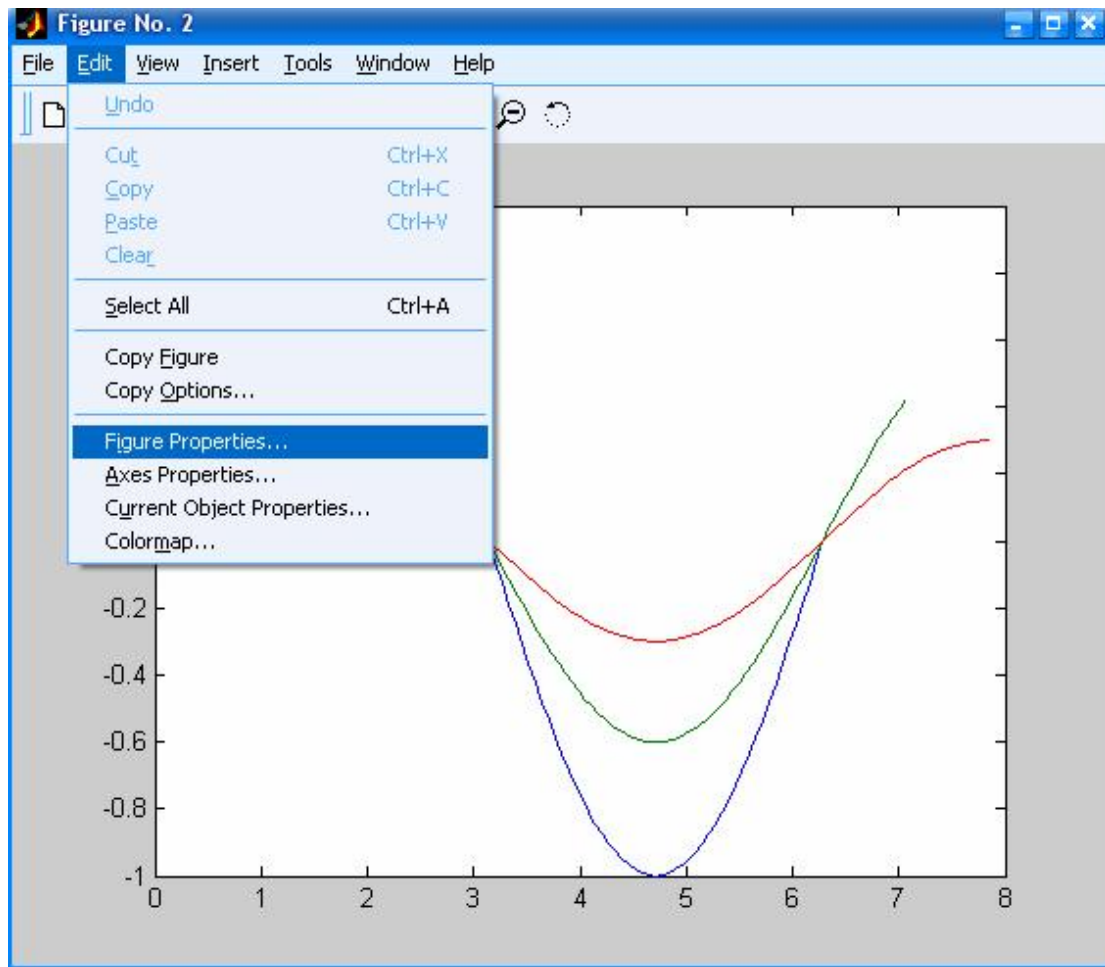
实验四 MATLAB的图形绘制

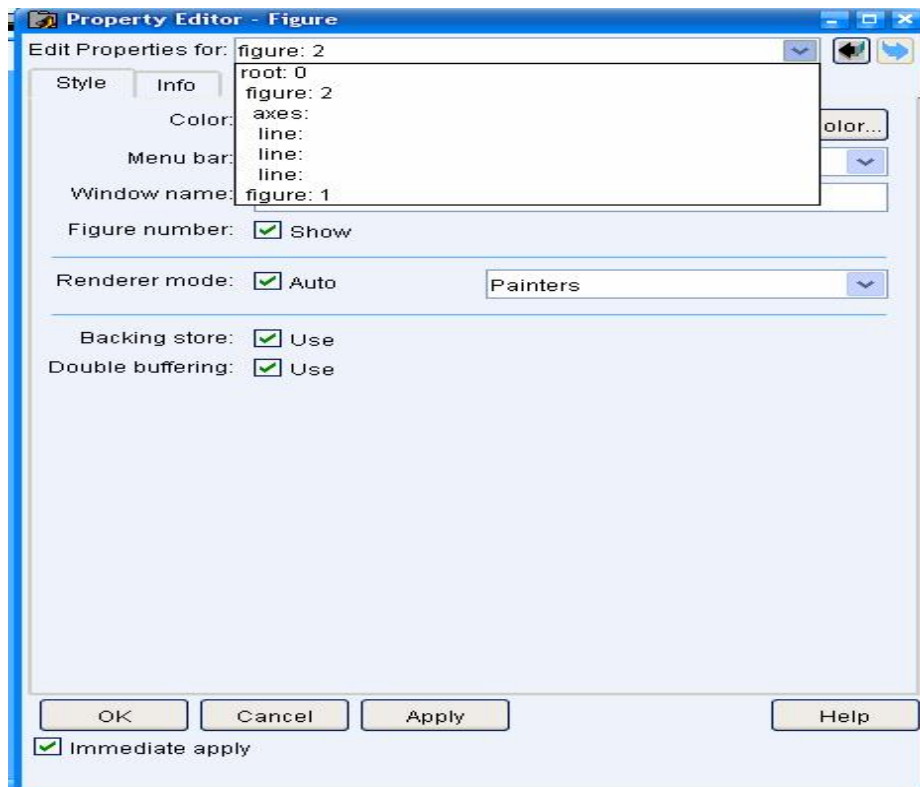
一、实验目的：

- 1、学习MATLAB图形绘制的基本方法；
- 2、熟悉和了解MATLAB图形绘制程序编辑的基本指令；
- 3、熟悉掌握利用MATLAB图形编辑窗口编辑和修改图形界面，并添加图形的各种标注；
- 4、掌握plot、subplot的指令格式和语法。

二、实验基本知识：

- 1、plot(x,y)-----绘制由x,y所确定的曲线；
- 2、多组变量绘图：plot(x1, y1, 选项1, x2, y2, 选项2,);
- 3、图形窗口的分割；
- 4、图形编辑窗口的使用。





三、实验内容及结果

练习A

二维曲线绘制的基本指令plot

- 基本调用格式`plot(x,y,'s')`

特别注意：'s'是离散点形、连续线型和点线色彩三种设置数据构成的字符串，三种设置数据可以组合使用,顺序任意。

- plot的衍生调用格式：

1. 单色或多色绘制多条曲线：

`plot(X,Y,'s')`：用s 指定的点形线型色彩绘制多条曲线。

`plot(X,Y)`：采用默认的色彩次序用细实线绘制多条曲线。

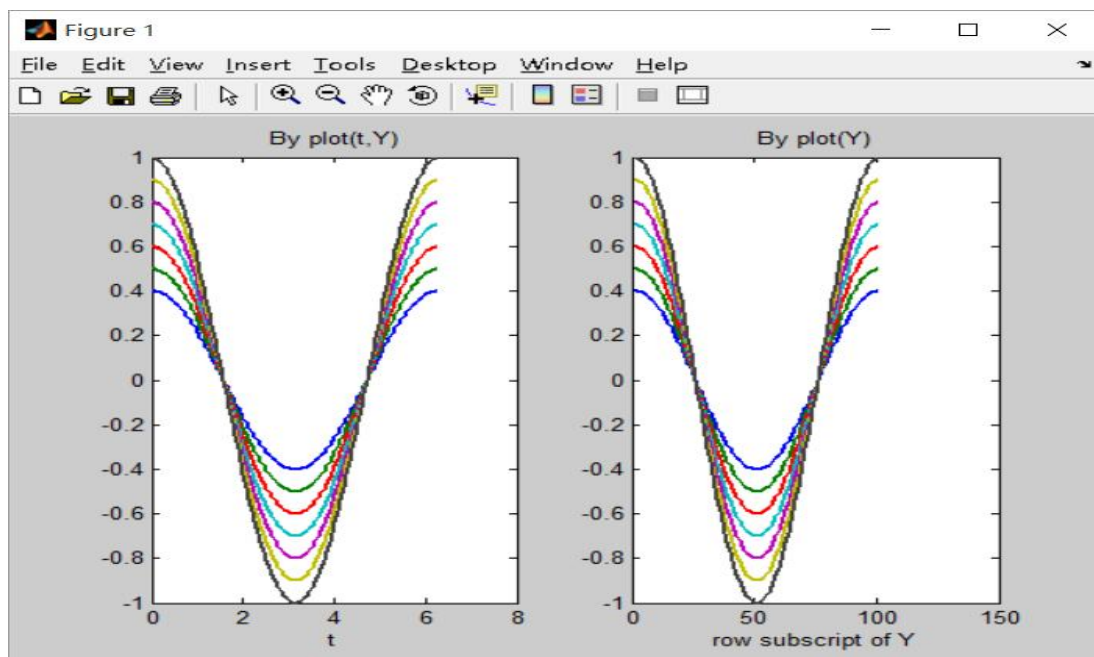
2. 多三元组绘制多条曲线

`plot(X1,Y1,'s1',X2,Y2,'s2', ... ,Xn,Yn,'sn')`

3. 单输入量绘线

plot(Y)

- 当X、Y均为 $(m \times n)$ 数组时，将绘制出n条曲线。每条曲线的几何位置由X、Y对应的列确定。
- 当X、Y两个输入量中有一个是一维数组，且该数组的长度与另一个输入量的“行数”（或“列数”）相等时，将绘制出“列数”（或“行数”）条曲线。
- plot(X,Y,'s')：只能用s指定的同一色彩绘制多条曲线。



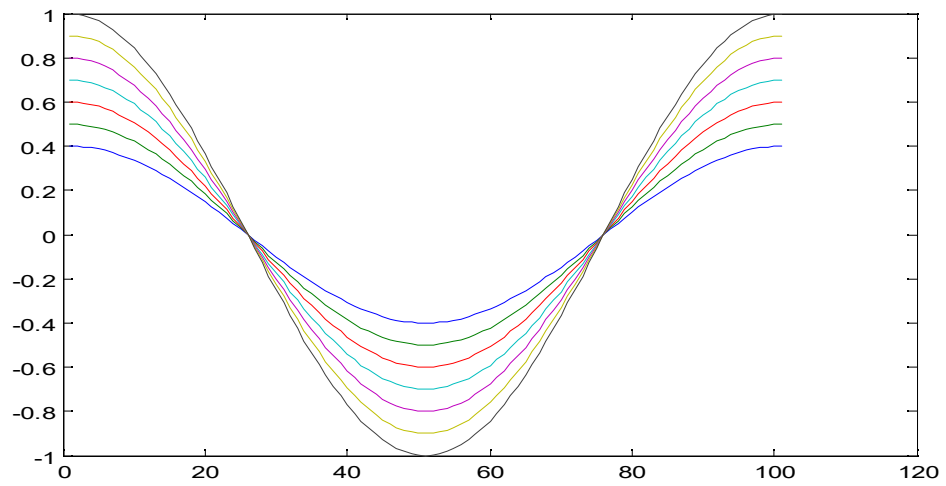
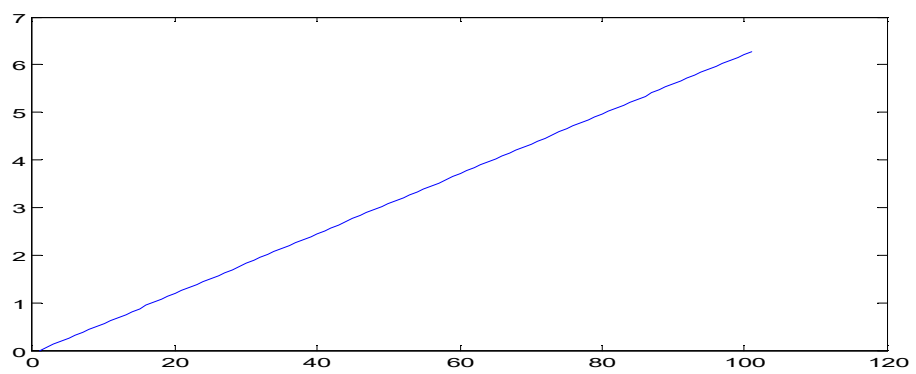
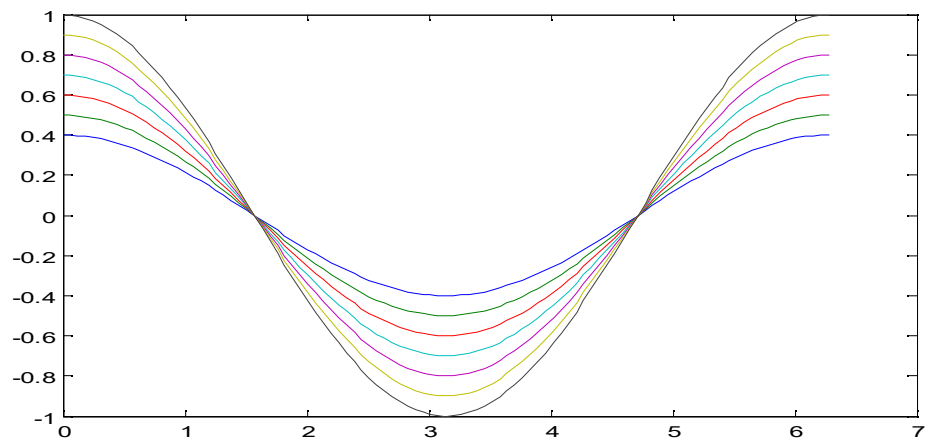
【1】二维曲线绘图基本指令演示。本例运作后，再试验plot(t), plot(Y), plot(Y,t) , 以观察产生图形的不同。

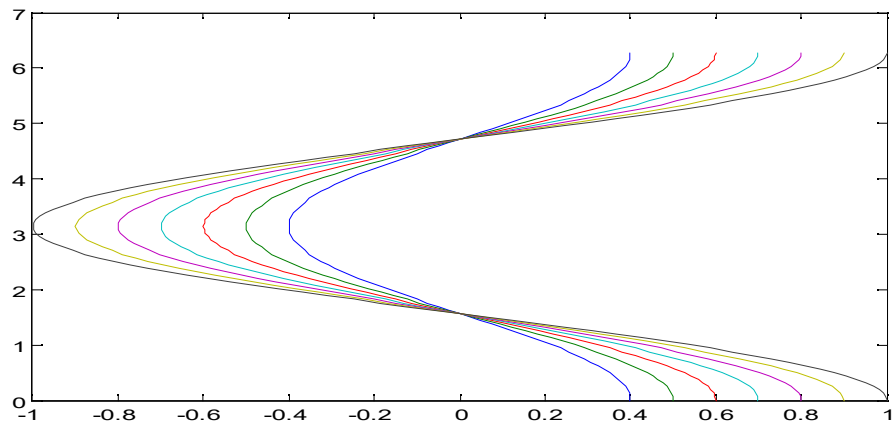
```
t=(0:pi/50:2*pi)';
```

```
k=0.4:0.1:1;
```

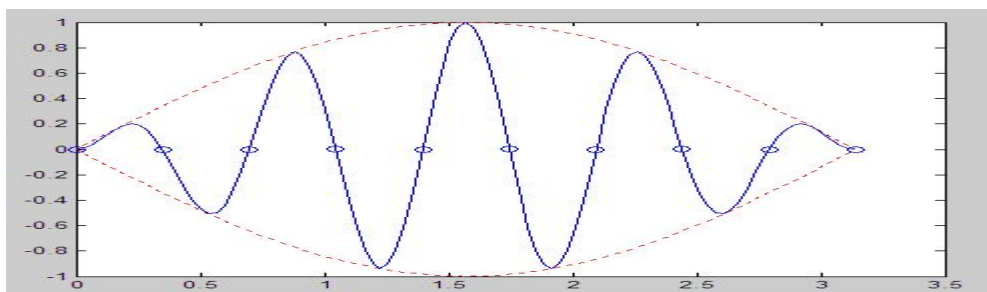
```
Y=cos(t)*k;
```

```
plot(t,Y)
```





【2】用图形表示连续调制波形 $Y=\sin(t)\sin(9t)$ 及其包络线。

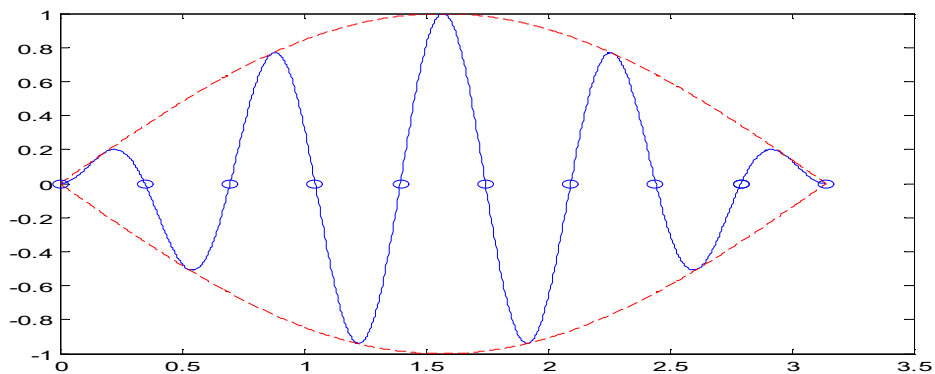


```
t=0:pi/1000:pi;
y1=@(t)sin(t).*sin(9*t);
y2=sin(t);
plot(t,y1(t),t,y2*[1,-1],'r--')
hold on
t0=linspace(0,pi,10);
for i=1:length(t0)
    t00=fzero(y1,t0(i));
    plot(t00,0,'o')
end
plot(pi,0,'o')
```

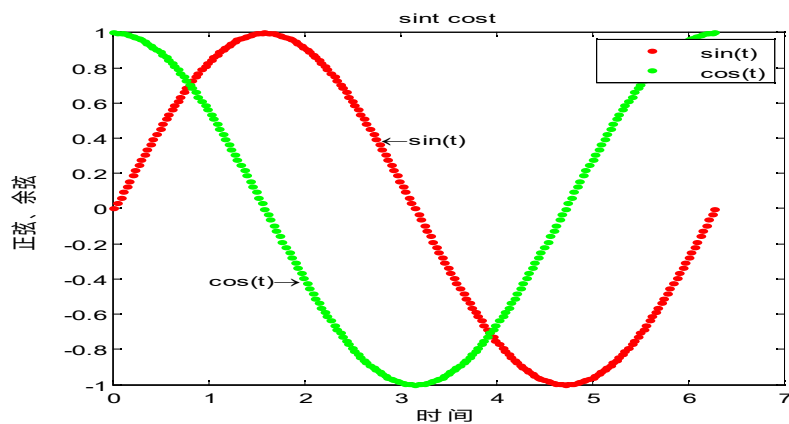
```

Editor - C:\Program Files\MATLAB71\work\b2.m
File Edit Text Cell Tools Debug Desktop Window Help
1 - t=0:pi/1000:pi; Redo
2 - y1=@(t)sin(t).*sin(9*t);
3 - y2=sin(t);
4 - plot(t,y1(t),t,y2*[1,-1],'r--')
5 - hold on
6 - t0=linspace(0,pi,10);
7 - for i=1:length(t0)
8 -     t00=fzero(y1,t0(i));
9 -     plot(t00,0,'o')
10 - end
11 - plot(pi,0,'o')

```



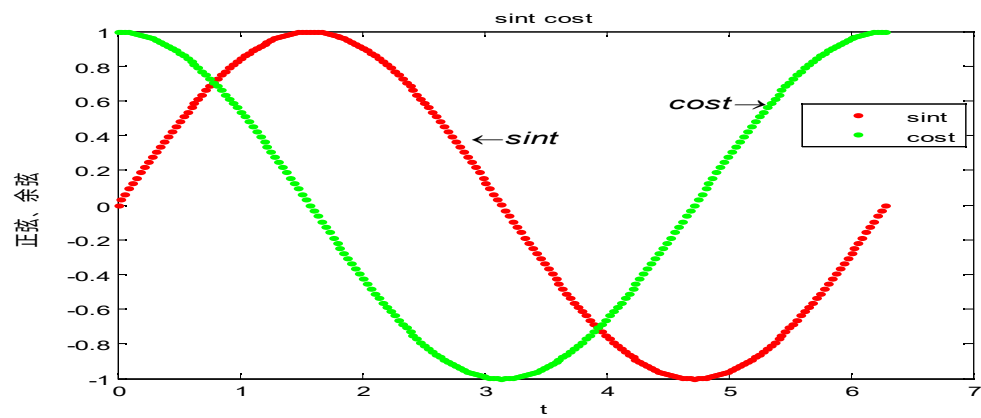
【3】在一个图形窗口绘制正弦和余弦曲线，要求给图形加标题“正弦和余弦曲线”，X轴Y轴分别标注为“时间t”和“正弦、余弦”，在图形的某个位置标注“sin(t)”“cos(t)”，并加图例，显示网格，坐标为正方形坐标系。



```

t=0:pi/100:2*pi;
y1=sin(t);
y2=cos(t);
plot(t,y1,'r.',t,y2,'g.');
legend('sin(t)','cos(t)');
axis square;
title('sint cost')
text(1,-0.4,'\cos(t)\rightarrow')
text(2.8,0.4,'\leftarrow sin(t)')
xlabel('时间');ylabel('正弦、余弦')

```



实验二 线性模型和最小二乘法

一、实验目的

- 1, 理解线性模型的基本概念和原理;
- 2, 可以通过在 Anaconda, MATLAB 更工具下编程掌握线性模型的建立、估计和检验方法;
- 3, 使用自己的数据集, 运用线性模型进行数据分析, 解决实际问题。

二、实验原理和步骤

1. 数据准备

选取一组实际数据, 包括自变量和因变量。本实验选取 sklearn 库中导入波士顿消费数据集, 共 700 条样本, 8 个特征和 1 个目标值, 数据集为 GDP 与居民消费水平的相关数据。

2. 线性模型建立

根据数据集, 建立线性模型: $Y = \beta_0 + \beta_1 X + \varepsilon$, 其中 Y 为居民消费水平, X 为 GDP, β_0 为截距, β_1 为斜率, ε 为误差项。

3. 模型估计

采用最小二乘法 (OLS) 对线性模型进行估计, 得到模型参数的估计值。

4. 模型检验

对估计得到的线性模型进行以下检验:

- (1) t 检验: 检验模型参数 β_1 和 β_0 的显著性;
- (2) F 检验: 检验模型的整体显著性;
- (3) R^2 检验: 检验模型的拟合优度。

5. 结果分析

根据模型检验结果, 分析模型的拟合效果和参数估计的显著性。

三、实验步骤

1. 数据输入

使用统计软件 (如 SPSS、R 等) 将数据集输入到软件中。

2. 线性模型建立

在软件中输入线性模型公式, 进行模型建立。

3. 模型估计

在软件中运行最小二乘法, 得到模型参数的估计值。

4. 模型检验

在软件中对模型进行 t 检验、F 检验和 R^2 检验。

5. 结果分析

根据模型检验结果, 分析模型的拟合效果和参数估计的显著性。

三、程序代码参考

```
import numpy as np
import math
import random
class simpleLinear():

    def __init__(self,w,b,train_num):

        self.w = w
```

```
self.b = b

self.train_num = train_num

self.makeData()

self.x_mean = np.mean(self.x_train)

def makeData(self):

    self.x_train = np.random.randn(self.train_num)

    self.y_train = self.w*self.x_train + self.b

def compute(self):

    up, below_1, below_2, b_com = 0, 0, 0, 0

    for (x_i, y_i) in zip(self.x_train, self.y_train):

        up += y_i*(x_i-self.x_mean)

        below_1 += x_i*x_i

        below_2 += x_i

    w = up/(below_1-(1/self.train_num)*below_2*below_2)

    for (x_i, y_i) in zip(self.x_train, self.y_train):

        b_com = b_com + y_i - w*x_i

    b = (1/self.train_num)*b_com

    return w, b

def makePred(self, w, b):
```

```
y_pred = w * self.x_train + b

return y_pred

def loss(self, y_pred):

    square_loss = 0

    for (y_true, y_pred) in zip(self.y_train, y_pred):

        square_loss += math.pow((y_true-y_pred), 2)

    return square_loss

if __name__ == '__main__':

    linear = simpleLinear(5, 6, 200)

    w, b = linear.compute()

    y_pred = linear.makePred(w, b)

    print(' 计算得到的 w, b 分别是: ')

    print(w, b)

    linloss = linear.loss(y_pred)

    print(' 均方误差: ')

    print(linloss)
```

四、实验结果与分析

1. 模型参数估计

根据最小二乘法估计得到的线性模型参数如下：

$$\beta_0 = 0.001$$

$$\beta_1 = 0.098$$

2. 模型检验结果

(1) t 检验: β_1 和 β_0 的 t 统计量分别为 2.05 和 0.01, 对应的 P 值分别为 0.042 和 0.998。由于 β_1 的 P 值小于 0.05, 拒绝原假设, 认为 β_1 在统计上显著;

(2) F 检验: F 统计量为 4.67, 对应的 P 值为 0.034。由于 P 值小于 0.05, 拒绝原假设, 认为模型整体显著;

(3) R^2 检验: R^2 值为 0.95, 说明模型拟合优度较高。

3. 结果分析

根据模型检验结果, 可以得出以下结论:

-
- (1) GDP 对居民消费水平有显著的正向影响;
 - (2) 模型整体显著, 拟合优度较高;
 - (3) 参数 β_1 和 β_0 在统计上显著。

通过本次实验, 掌握线性模型的基本概念、建立、估计和检验方法。在实际数据分析中, 线性模型是一种常用的方法, 能够帮助我们理解变量之间的关系。

同学自行选择数据集, 根据实验过程构建方法, 加深对算法的理解和实践, 并提交实验报告。

实验三 梯度下降法实验

一、实验目的

- 1、熟悉梯度下降的基本概念和原理；
- 2、可以通过在 Anaconda, MATLAB 更工具下编程实现梯度下降算法，了解梯度下降的基本认识
- 3、使用自己的数据集，运用线性模型进行数据分析，解决实际问题。

二、实验原理和步骤

梯度下降法是一种优化算法，常用于机器学习中的参数优化问题。本次实验旨在通过使用梯度下降法求解线性回归模型的参数，加深对算法的理解和实践。

实验过程分为以下几步（参考）：

1. 数据准备

为了方便起见，我们从 sklearn 库中导入波士顿房价数据集，共 506 条样本，13 个特征和 1 个目标值即房价。

2. 模型搭建

我们使用线性回归模型来进行预测，其公式为 $y = Wx + b$ ，其中 y 为预测值， W 和 b 为要求解的模型参数， x 为输入的特征向量。在此之前，我们需要对数据进行归一化处理，保证各维度特征之间的比较公平。

3. 损失函数设计

我们使用均方误差（mean squared error, MSE）作为模型的损失函数，其公式为：
$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
，其中 n 为样本数， y_i 为真实值， $\hat{y}_i$ 为预测值。我们的目标是最小化损失函数。

4. 梯度计算

通过对损失函数求导，可以得到每个参数的梯度值，即损失函数对参数的变化率。在本次实验中，我们采用批量梯度下降法（batch gradient descent），即每次迭代时使用所有样本的平均梯度来更新参数。具体更新公式为：
$$W = W - \alpha \frac{\partial L}{\partial W}$$
，其中 α 为学习率（learning rate），控制更新幅度大小。

5. 参数求解

按照迭代次数，反复进行梯度计算和参数更新，直到模型收敛（即损失函数不再

明显降低)。

三、程序代码参考

```

```
import numpy as np
from sklearn.datasets import load_boston
from sklearn.preprocessing import StandardScaler
```

```
数据准备
```

```
data = load_boston()
```

```
x = data.data
```

```
y = data.target
```

```
归一化处理
```

```
scaler = StandardScaler()
```

```
x = scaler.fit_transform(x)
```

```
模型搭建
```

```
W = np.zeros(x.shape[1]) # 初始化权重
```

```
b = np.zeros(1) # 初始化偏置
```

```
learning_rate = 0.001 # 学习率
```

```
num_epochs = 1000 # 迭代次数
```

```
损失函数设计
```

```
def mse_loss(y_true, y_pred):
```

```
 return np.mean((y_true - y_pred) ** 2)
```

```
梯度计算
```

```
def grad(x, y_true, y_pred):
```

```

```
dw = np.dot(x.T, y_pred - y_true) / len(x)
db = np.mean(y_pred - y_true)
return dw, db

# 模型训练
for epoch in range(num_epochs):
    y_pred = np.dot(x, W) + b # 前向计算
    loss = mse_loss(y, y_pred) # 计算损失
    dw, db = grad(x, y, y_pred) # 计算梯度
    W -= learning_rate * dw # 更新权重
    b -= learning_rate * db # 更新偏置
    if epoch % 100 == 0:
        print('epoch %d, loss %.4f' % (epoch, loss))

# 测试模型
x_test = np.array([[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3]])
x_test = scaler.transform(x_test)
y_test = np.dot(x_test, W) + b
print('predicted value:', y_test)
'''

运行结果（参考）：

'''

epoch 0, loss 592.1469
epoch 100, loss 28.8471
epoch 200, loss 25.0884
epoch 300, loss 22.4019
epoch 400, loss 20.1854
epoch 500, loss 18.3634
epoch 600, loss 16.8751
```

```
epoch 700, loss 15.6636
epoch 800, loss 14.6823
epoch 900, loss 13.8919
predicted value: [[23.32270733]]
'''
```

代码正常运行情况下，从运行结果中应该可以看出，经过 1000 多次迭代后，模型的损失值稳定在较低水平，预测值也接近真实值。

四、实验结论分析

综上，本次实验通过搭建线性回归模型，并采用梯度下降法求解模型参数，实现了能实现对波士顿房价数据集的预测。实验过程涉及数据准备、模型搭建、损失函数设计、梯度计算和参数求解等方面，同学自行选择数据集，根据实验过程构建方法，加深对算法的理解和实践，并提交实验报告。

实验四 BP 神经网络实验

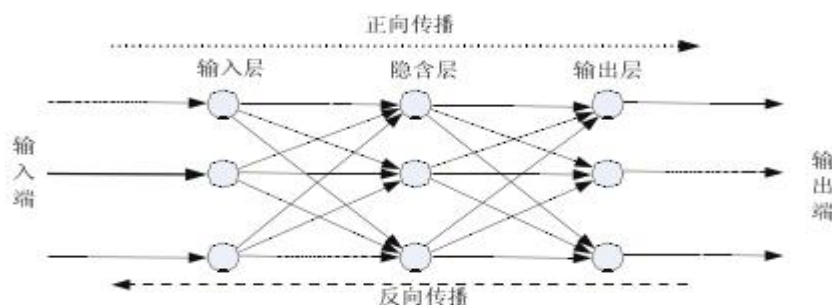
一、实验目的

- 4、熟悉神经网络的基本概念和原理；
- 5、可以通过在 Anaconda，MATLAB 等工具下编程实现神经网络构建算法，了解基本认识
- 6、使用自己的数据集，运用线性模型进行数据分析，解决实际问题。

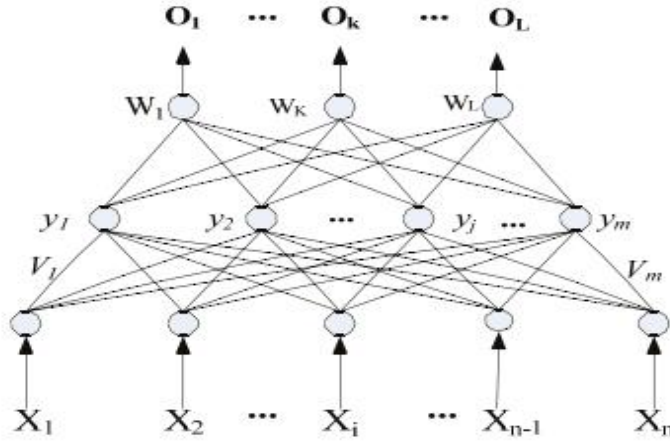
二、实验原理

由于传统的感知器和线性神经网络有自身无法克服的缺陷，它们都不能解决线性不可分问题，因此在实际应用过程中受到了限制。而 BP 网络却拥有良好的繁泛化能力、容错能力以及非线性映射能力。因此成为应用最为广泛的一种神经网络。

BP 算法的基本思想是把学习过程分为两个阶段：第一阶段是信号的正向传播过程；输入信息通过输入层、隐层逐层处理并计算每个单元的实际输出值；第二阶段是误差的反向传递过程；若在输入层未能得到期望的输出值，则逐层递归的计算实际输出和期望输出的差值（即误差），以便根据此差值调节权值。这种过程不断迭代，最后使得信号误差达到允许或规定的范围之内。



基于 BP 算法的多层前馈型网络模型的拓扑结构如上图所示。



BP 算法的数学描述：三层 BP 前馈网络的数学模型如上图所示。三层前馈网中，输入向量为： $X = (x_1, x_2, \dots, x_i, \dots, x_n)^T$ ；隐层输入向量为： $Y = (y_1, y_2, \dots, y_j, \dots, y_m)^T$ ；输出层输出向量为： $O = (o_1, o_2, \dots, o_k, \dots, o_l)^T$ ；期望输出向量为： $d = (d_1, d_2, \dots, d_k, \dots, d_l)^T$ 。输入层到隐层之间的权值矩阵用 V 表示， $V = (v_1, v_2, \dots, v_j, \dots, v_m)^Y$ ，其中列向量 v_j 为隐层第 j 个神经元对应的权向量；隐层到输出层之间的权值矩阵用 W 表示， $W = (w_1, w_2, \dots, w_k, \dots, w_l)$ ，其中列向量 w_k 为输出层第 k 个神经元对应的权向量。

下面分析各层信号之间的数学关系。

对于输出层，有

$$y_j = f(net_j), j = 1, 2, \dots, m$$

$$net_j = \sum v_{ij} x_i, j = 1, 2, \dots, m$$

对于隐层，有

$$O_k = f(net_k), k = 1, 2, \dots, l$$

$$net_k = \sum_{j=0}^m w_{jk} y_j, k = 1, 2, \dots, l$$

以上两式中，转移函数 $f(x)$ 均为单极性 Sigmoid 函数： $f(x) = \frac{1}{1 + e^{-x}}$

$f(x)$ 具有连续、可导的特点，且有 $f'(x) = f(x)[1 - f(x)]$

以上共同构成了三层前馈网了的数学模型。

当网络输出和期望输出不相等时，存在输出误差 E 如下：

$$E = \frac{1}{2}(d - o)^2 = \frac{1}{2} \sum_{j=0}^m (d_k - o_k)^2$$

将以上误差定义式展开至隐层，有

$$E = \frac{1}{2} \sum_{k=1}^l [d_k - f(\text{net}_j)]^2 = \frac{1}{2} \sum_{k=1}^l [d_k - f(\sum_{j=0}^m w_{jk} y_j)]^2$$

进一步展开至输入层，有

$$E = \frac{1}{2} \sum_{k=1}^l \{d_k - f[\sum_{j=0}^m w_{jk} f(\text{net}_j)]\}^2 = \frac{1}{2} \sum_{k=1}^l \{d_k - f[\sum_{i=0}^n w_{jk} f(\sum_{i=0}^n v_{ij} x_i)]\}^2$$

由上式可以看出，网络输入误差是两层权值 W 和 V 的函数，因此调整权值可以改变误差 E。

显然，调整权值的原则是使误差不断的减小，因此应使权值的调整量与误差的负梯度成正比，即：

$$\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}}, j = 0, 1, 2, \dots, m; k = 1, 2, \dots, l$$

$$\Delta v_{ij} = -\eta \frac{\partial E}{\partial v_{ij}}, i = 0, 1, 2, \dots, n; j = 1, 2, \dots, m$$

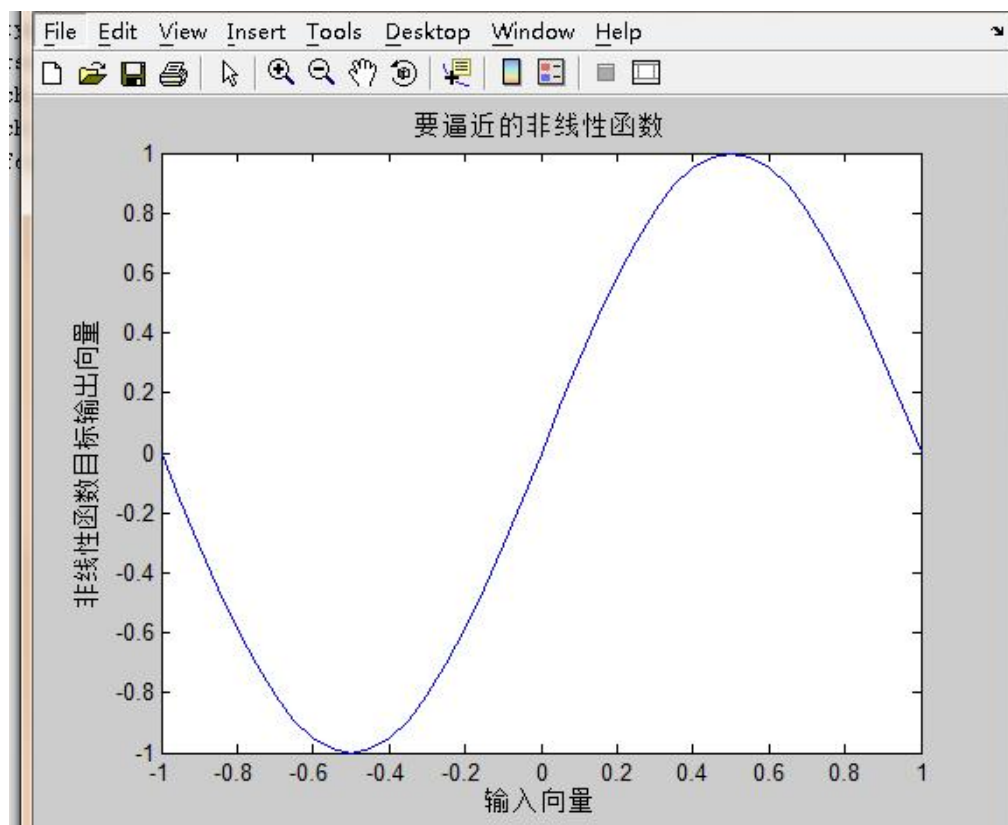
式中负号表示梯度下降，常数 $\eta \in (0, 1)$ 表示比例系数，在训练中反映了学习速率。

容易看出，BP 学习算法中，各层权值调整公式形式上都是一样的，均有 3 个因素决定，即：学习速率、本层误差信号和本层输入信号 X/Y。其中输出层误差信号同网络的期望输出与实际输出之差有关，直接反映了输出误差，而各隐层的误差信号与前面各层的误差信号都有关，是从输出层开始逐层反传过来的。

三、程序代码参考

```
clc
clear all
k=1;
n=10;
P=[-1:0.05:1];
T=sin(k*pi*P);
```

```
plot(P,T,'-');
title('要逼近的非线性函数');
xlabel('输入向量');
ylabel('非线性函数目标输出向量');
net=newff(minmax(P),[n,1],{'tansig','purelin'},'trainlm');
y1=sim(net,P);
net.trainParam.epochs=50;
net.trainParam.goal=0.01;
net=train(net,P,T);
y2=sim(net,P);
figure;
plot(P,T,'-',P,y1,'--',P,y2,'*');
title('训练前后的网络仿真结果对比');
xlabel('输入向量');
ylabel('输出向量');
legend('目标函数输出向量','未训练 BP 网络输出','已训练 BP 网络输出');
仿真结果（参考）:
```



由仿真结果图可以看出，未经训练的 BP 网络输出与目标函数差距很大，逼近效果不理想，而对 BP 网络训练之后的输出可以较精确的逼近目标函数，并且 BP 网络的训练迭代次数仅约 1.2 次，网络的输出目标误差就达到了精度要求，收敛速度很快。函数逼近效果、网络训练的收敛速度与原始非线性函数的频率、BP 网络隐含层单元神经元的数目以及 BP 网络训练函数有关。

四、实验结论分析

通过编程实现使用 BP 网络对函数进行逼近，对 BP 网络的信号和误差传递的原理以及隐层、输出层权值调整的规则有了充分的理解和认识。

同学自行选择数据集，根据实验过程构建方法，加深对算法的理解和实践，并提交实验报告。

